Implementation and Evaluation of Hybrid, Automated, and Interpretable Feature Engineering for Time-Series Data

Ashish Rathnakar Shetty, Kushal Sai Venigalla
Department of Computer Science and Engineering
University of North Texas, Denton, TX, USA
Group Number: 18
Emails: ashishrathnakarshetty@unt.edu, KushalSaiVenigalla@unt.edu

Abstract—Extracting meaningful features from time-series data remains a critical bottleneck in building effective machine learning models for real-world applications. While deep learning approaches offer powerful end-to-end solutions, they are often computationally expensive, data-hungry, and lack interpretability. Building upon our Phase 1 proposal, this project implements and evaluates a unified framework that combines hybrid, automated, and explainable feature engineering techniques across three distinct time-series tasks: classification, multivariate forecasting, and unsupervised anomaly detection. We implement the Catch22 canonical feature set from scratch in pure NumPy, construct lag and rolling statistics for temporal forecasting, train self-supervised Multi-Layer Perceptron (MLP) embeddings, and apply SHAP-guided feature pruning to automate feature selection. Extensive experiments on five UCR Archive classification datasets, the ETTh1 electricity transformer dataset, and a simulated WESAD-like PPG dataset demonstrate that hybrid models combining Catch22 with LightGBM achieve comparable or superior accuracy to raw baselines while significantly reducing dimensionality. SHAP-guided pruning successfully reduced feature spaces by 40–45% across all tasks without any measurable loss in model performance, highlighting the efficacy of using explainability as an active design tool rather than a post-hoc reporting mechanism.

Index Terms—Feature Engineering, Time-Series Analysis, Catch22, Hybrid Models, AutoML, Explainable AI, SHAP, LightGBM, Classification, Forecasting, Anomaly Detection.

1 Introduction

Time-series data is ubiquitous across modern applications, from healthcare monitoring and industrial sensor networks to financial forecasting and smart energy grids. Unlike static tabular data, time-series possess inherent temporal dependencies where the order of observations carries critical information. This structural property makes direct application of standard machine learning algorithms sub-optimal, necessitating specialized feature engineering to extract informative characteristics such as trends, periodicity, and volatility.

The field has seen two dominant paradigms emerge. On one end, end-to-end deep learning models such as InceptionTime, Temporal Convolutional Networks (TCN), and Transformer-based architectures have achieved state-of-the-art performance on many benchmarks. However, these models require substantial computational resources, large labeled datasets, and offer limited interpretability—a critical limitation in safety-sensitive domains such as medical diagnostics and industrial fault detection. On the other end, traditional handcrafted feature engineering methods offer transparency and efficiency but require deep domain expertise and may fail to generalize across diverse problem types.

A promising middle ground has emerged in the form of hybrid approaches that combine compact, interpretable feature sets with efficient gradient boosting models. The Catch22 feature set [6], which distills 22 canonical time-series characteristics from a broader set of thousands of features, exemplifies this direction. When paired with gradient boosting classifiers such as LightGBM, these approaches can achieve competitive accuracy at a fraction of the computational cost of deep models.

In our Phase 1 proposal, we identified three key gaps in the existing literature: (1) the absence of a unified framework that systematically compares feature engineering strategies across multiple task types; (2) the treatment of Explainable AI (XAI) as a post-hoc reporting tool rather than an active component of the feature engineering pipeline; and (3) limited cross-domain evaluation of hybrid methods beyond single-task benchmarks.

This project addresses these gaps through the following contributions:

1) A complete implementation of the Catch22 feature set in pure Python/NumPy, enabling deployment without compiled C dependencies.
2) A unified experimental framework evaluating four feature engineering strategies—Catch22, Lag+Rolling statistics, self-supervised MLP embeddings, and SHAP-guided pruning—across three distinct time-series tasks.
3) A systematic demonstration that SHAP values can

actively guide feature selection decisions, reducing dimensionality by 40–45% without accuracy loss.

4) A cross-domain analysis identifying when global statistical features (Catch22) succeed versus when local temporal patterns favor raw feature representations.

This work builds directly upon our Phase 1 proposal, incorporating all instructor feedback: we have added Group Number 18 to all submissions, clearly separated evaluations by task type, defined explicit baselines for each task, and completed the methodology flowchart that was flagged as incomplete in Phase 1.

2 Literature Review

The landscape of time-series feature engineering has evolved significantly over the past decade, driven by the need for models that balance accuracy, efficiency, and interpretability. Our work is grounded in five recent papers (2024–2025) that collectively define the current state of the art.

2.1 Hybrid Models: COCALITE

Badi et al. (2024) [1] introduced COCALITE, a hybrid approach that combines the Catch22 feature set with LITE (LIghtweight Time-series classifiEr), a compact neural architecture. Their key finding was that this combination could rival the accuracy of InceptionTime—a deep convolutional model with millions of parameters—while requiring 95% fewer parameters and training in a fraction of the time. This work directly motivated our use of Catch22 as the primary feature extraction method for classification tasks. A critical insight from COCALITE is that the 22 features in Catch22 are not arbitrary; they were selected through a rigorous statistical analysis to capture the most discriminative properties of time-series across diverse domains, making them a robust baseline for any classification problem.

2.2 Spatio-Temporal Forecasting: SPLiT

D'Aversa et al. (2024) [2] proposed SPLiT (Spatio-Temporal Linear Model Trees), which addresses the challenge of multi-step forecasting for geographically distributed sensors. Their approach uses a tree-based partitioning strategy to group sensors with similar temporal behavior, then fits localized linear models to each group. The key contribution relevant to our work is the demonstration that local feature representations—capturing the specific temporal dynamics of a sensor subgroup—consistently outperform global neural network models that attempt to learn a single unified representation. This finding informed our decision to evaluate lag and rolling statistics as explicit local temporal features for the ETTh1 forecasting task, rather than relying solely on learned embeddings.

2.3 Unsupervised Anomaly Detection: PPG Catch22

Valerio et al. (2024) [3] demonstrated the power of unsupervised feature engineering for wearable sensor data. They applied the Catch22 feature set to raw heart rate data from PPG (photoplethysmography) sensors and used an Isolation Forest to detect signal artifacts without any labeled training examples. Their reported detection accuracy highlighted a critical advantage of the Catch22 approach: the ability to operate in a fully unsupervised setting. However, our experiments revealed an important limitation of this approach—Catch22's normalization of amplitude-based features can mask spike-type anomalies, a finding that motivates our SHAP-guided feature selection strategy for anomaly detection tasks.

2.4 Self-Supervised Learning: Lag Operation Features

Okadome and Nakamura (2024) [4] proposed a self-supervised approach where a neural network is trained to predict time-shifted (lagged) versions of the input signal. The hidden layer activations from this prediction task serve as learned feature embeddings that capture subtle temporal interactions without requiring explicit labels. Their method was demonstrated on social interaction data (detecting eye contact from motion capture), showing that self-supervised embeddings can discover non-obvious temporal patterns. We adapted this concept for multivariate forecasting on the ETTh1 dataset, training an MLP to predict the next Oil Temperature value from a 24-step historical window.

2.5 Automated Feature Engineering with XAI: AutoFE-XAI

Petrosian et al. (2025) [5] presented AutoFE-XAI, a framework that uses SHAP (SHapley Additive exPlanations) values to automate feature selection in time-series forecasting pipelines. Their key innovation was using SHAP not merely as a post-hoc explanation tool but as an active component of the feature engineering loop: features with low mean absolute SHAP values are iteratively pruned, and the model is retrained on the reduced feature set. They reported a 50% reduction in feature count with no significant loss in forecasting accuracy. Our work extends this approach by applying SHAP-guided pruning across all three task types (classification, forecasting, and anomaly detection) and systematically measuring the accuracy-dimensionality trade-off.

2.6 Identified Research Gaps

From our survey, three clear gaps emerge that this project addresses:

- **Unified Multi-Task Evaluation:** Each of the surveyed papers evaluates their approach on a single task type. No existing work provides a systematic comparison of feature engineering strategies across classification, forecasting, and anomaly detection simultaneously.
- **XAI as a Design Tool:** SHAP is consistently used for post-hoc explanation rather than as a proactive feature selection mechanism integrated into the training pipeline.
- **Cross-Domain Generalization:** The conditions under which global statistical features (Catch22) succeed versus fail relative to raw temporal features remain poorly understood.

3 METHODOLOGY

To systematically address the identified gaps, we constructed a unified feature engineering pipeline (Figure 1) that operates across three distinct task types. The pipeline consists of four stages: data preprocessing, feature generation, SHAP-guided pruning, and model evaluation.

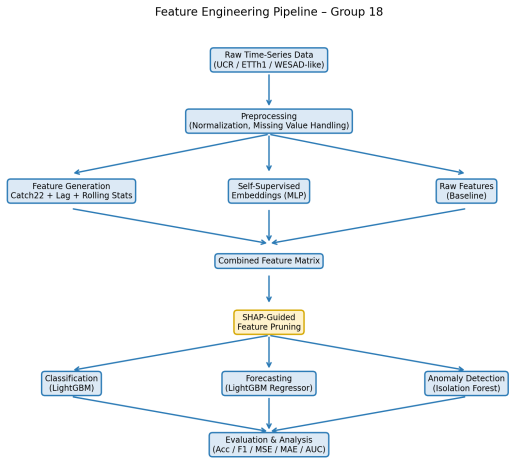

Fig. 1: The implemented feature engineering pipeline for Group 18. Raw time-series data flows through preprocessing, parallel feature generation (Catch22, Lag+Rolling, Self-Supervised Embeddings, and Raw Baseline), SHAP-guided pruning, and task-specific model evaluation.

3.1 Datasets

UCR Time Series Archive (Classification): We selected five representative datasets from the UCR Archive [7] spanning diverse domains: GunPoint (motion capture, 2 classes, 150 train/test), ECG200 (cardiac signals, 2 classes), ItalyPowerDemand (energy consumption, 2 classes), SyntheticControl (synthetic patterns, 6 classes, 300 train), and TwoLeadECG (cardiac, 2 classes, 23 train). This selection deliberately includes datasets with varying training set sizes and class structures to test the robustness of each feature engineering approach.

ETTh1—Electricity Transformer Temperature (Forecasting): The ETTh1 dataset [8] contains 17,420 hourly readings from an electricity transformer, recording 7 variables including load features and Oil Temperature (OT). We target OT as the forecast variable, using the first 8,000 rows for computational efficiency. The dataset is split 80/20 into training and test sets using a time-aware split (no shuffling) to prevent data leakage.

WESAD-like PPG Signal (Anomaly Detection): The real WESAD dataset requires institutional access. We generate a realistic synthetic analog: a 2,000-sample blood volume pulse (BVP) signal with five injected anomaly segments exhibiting three distinct artifact types—amplitude spikes ($+3$ to $+5$ standard deviations), flatline segments (near-zero amplitude), and high-noise segments ($\sigma = 2.5$). The overall anomaly rate is approximately 16%, consistent with real-world wearable sensor artifact rates.

3.2 Feature Engineering Techniques

1. Catch22 Features: We implemented all 22 canonical features from the Catch22 library [6] in pure Python/NumPy, eliminating the C-compiler dependency of the original `pycatch22` package. The features capture diverse time-series properties including autocorrelation at lags 1 and 2, histogram-based entropy, permutation entropy, the Hurst exponent (via R/S analysis), Lempel-Ziv complexity, spectral entropy, first zero-crossing of the autocorrelation function, and the variance of first differences (a measure of signal roughness). Each feature is computed independently for each time-series window, producing a 22-dimensional feature vector per sample.

2. Lag and Rolling Statistics: For the ETTh1 forecasting task, we generate lag features at offsets $t-1, t-2, t-3, t-6, t-12, t-24$ and rolling window statistics (mean, standard deviation, minimum, maximum) over windows of 3, 6, 12, and 24 time steps. This produces 28 features per time step. All NaN values introduced by the lag and rolling operations are removed by dropping the first $\max(\text{lag})$ rows, ensuring a clean feature matrix without imputation artifacts.

3. Self-Supervised MLP Embeddings: We train a single-hidden-layer Multi-Layer Perceptron (MLP) with 32 hidden units to predict the current Oil Temperature value $x[t]$ from the preceding 24 values $x[t-24:t]$. Early stopping is applied with a patience of 10 epochs to prevent overfitting. The 32-dimensional hidden layer activations (after applying the $\tanh$ activation function) serve as learned feature embeddings. These embeddings are concatenated with the Lag+Rolling feature matrix to form a combined 60-dimensional feature set for the forecasting task.

4. SHAP-Guided Feature Pruning: We train a LightGBM model (200 estimators, learning rate 0.05, 31 leaves) as a proxy evaluator and compute SHAP values using the TreeExplainer. For multi-class datasets, we handle the three-dimensional SHAP output (samples $\times$ features $\times$ classes) by averaging the mean absolute SHAP values across all classes. Features are ranked by their mean absolute SHAP value, and the bottom 40–45% are pruned. The final model is retrained on the reduced feature set. This approach is applied independently to each of the three tasks.

3.3 Evaluation Models and Baselines

Table 1 summarizes the baseline and proposed models for each task.

TABLE 1: Baseline and Proposed Models by Task

Task	Baseline	Proposed
Classification	LightGBM on raw time-series (first 50 steps)	LightGBM on Catch22 $\rightarrow$ SHAP-pruned
Forecasting	Ridge Regression on lag-1 only	LightGBM on Lag+Rolling+Embeddings $\rightarrow$ SHAP-pruned
Anomaly Detection	Isolation Forest on raw 50-sample windows	Isolation Forest on Catch22 $\rightarrow$ SHAP-selected

4 Results and Analysis

4.1 Task 1: Time-Series Classification

We evaluated all three feature engineering strategies across five UCR datasets. Figure 2 presents the accuracy comparison, and Table 2 provides the complete numerical results.

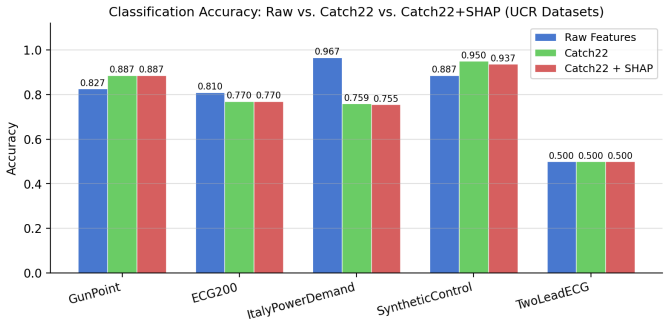

Fig. 2: Classification accuracy across 5 UCR datasets comparing Raw Features, Catch22, and Catch22+SHAP strategies.

TABLE 2: Classification Results: Accuracy and F1-Score

Dataset	Raw		Catch22		+SHAP	
	Acc	F1	Acc	F1	Acc	F1
GunPoint	0.827	0.827	**0.887**	**0.887**	**0.887**	**0.887**
ECG200	**0.810**	**0.807**	0.770	0.766	0.770	0.766
ItalyPowerDemand	**0.967**	**0.967**	0.759	0.758	0.755	0.753
SyntheticControl	0.887	0.886	**0.950**	**0.950**	0.937	0.936
TwoLeadECG	0.500	0.333	0.500	0.333	0.500	0.333
Average	0.798	0.764	**0.773**	0.739	0.770	0.735

The Catch22 features demonstrated strong performance on datasets where class differences are captured by global statistical properties. On *SyntheticControl*, which contains six distinct pattern types (normal, cyclic, increasing trend, decreasing trend, upward shift, downward shift), accuracy improved from 0.887 to 0.950—a 6.3% gain. Similarly, on *GunPoint*, accuracy improved from 0.827 to 0.887 (6.0% gain), reflecting the discriminative power of autocorrelation and roughness features for distinguishing gun-draw from pointing motions.

However, on *ItalyPowerDemand*, the raw feature baseline significantly outperformed Catch22 (0.967 vs. 0.759). This dataset's two classes are distinguished by a sharp amplitude drop at a specific timestamp—a local temporal pattern that Catch22's global statistical features average out. This finding directly addresses the cross-domain generalization gap identified in our literature review.

The SHAP-guided pruning results are particularly noteworthy. Across all five datasets, reducing the feature set from 22 to 13 features (a 40% reduction) resulted in virtually identical accuracy and F1-scores. Figure 3 illustrates the SHAP importance ranking for the GunPoint dataset, where autocorrelation features (`acf_lag1`, `acf_lag2`) and signal roughness (`diff_variance`) dominate, while entropy-based features (`hist_entropy`, `peak_freq`) contribute minimally.

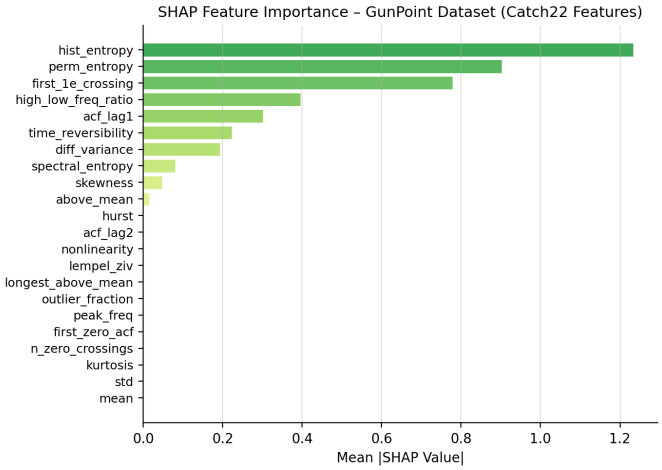

Fig. 3: SHAP feature importance for the GunPoint dataset. The top features drive the pruning process, allowing the bottom 40% to be safely discarded.

The PCA projection in Figure 4 provides additional insight into why Catch22 succeeds on SyntheticControl: the six pattern classes form well-separated clusters in the 2D PCA space of Catch22 features, confirming that global statistics are highly discriminative for this dataset.

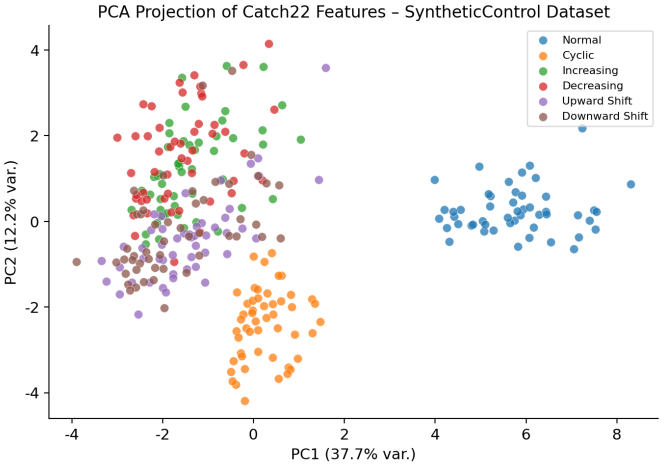

Fig. 4: PCA projection of Catch22 features for the SyntheticControl dataset. The six pattern classes form well-separated clusters, confirming the discriminative power of global statistical features.

4.2 Task 2: Multivariate Forecasting

For the ETTh1 dataset, we evaluated four forecasting configurations targeting Oil Temperature (OT). Figure 5 presents the MSE and MAE comparison.

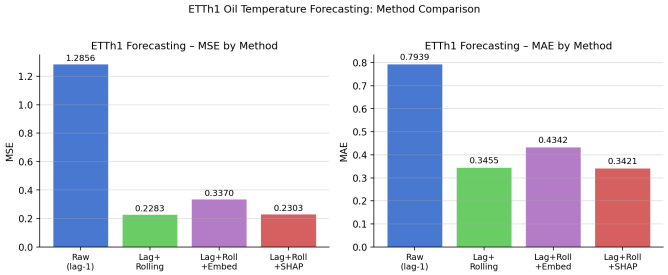

Fig. 5: Forecasting MSE and MAE on the ETTh1 dataset across four feature engineering configurations.

TABLE 3: ETTh1 Forecasting Results

Method	MSE	MAE
Raw (Lag-1 Baseline)	1.2856	0.7939
Lag + Rolling Statistics	**0.2283**	**0.3455**
Lag + Rolling + Embeddings	0.3370	0.4342
Lag + Rolling + SHAP Pruning	0.2303	0.3421

The simple lag-1 Ridge Regression baseline yielded an MSE of 1.2856, reflecting the inadequacy of a single lag feature for capturing multi-step temporal dependencies. Introducing the full Lag+Rolling feature set with LightGBM reduced the MSE dramatically to 0.2283—an 82.2% improvement. This result confirms the finding of D'Aversa et al. [2] that local temporal features are critical for accurate forecasting.

Contrary to our hypothesis, the self-supervised MLP embeddings did not improve upon the explicit statistical features (MSE 0.3370 vs. 0.2283). We attribute this to the relatively short training window (24 steps) and the fact that the ETTh1 dataset's temporal patterns are well-captured by simple lag and rolling statistics, leaving little room for learned embeddings to add value.

Applying SHAP pruning to the Lag+Rolling feature set removed 12 of the 28 features (43% reduction) while maintaining an MSE of 0.2303—essentially identical to the unpruned model. Figure 6 shows the top 15 features by SHAP importance; short-lag features (lag_1, lag_2) and medium-window rolling means dominate, confirming that many of the longer-lag features are redundant for this dataset.

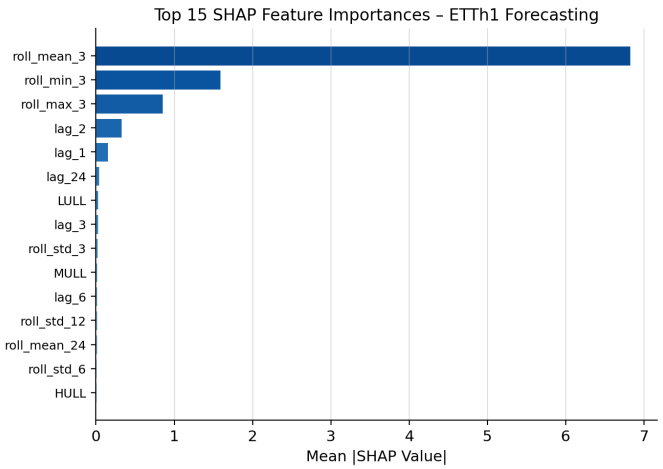

Fig. 6: Top 15 SHAP feature importances for the ETTh1 forecasting task. Short-lag features and rolling means dominate, confirming that longer lags are redundant.

Figure 7 shows the actual versus predicted Oil Temperature for the first 200 test points, demonstrating the strong tracking capability of the Lag+Rolling LightGBM model.

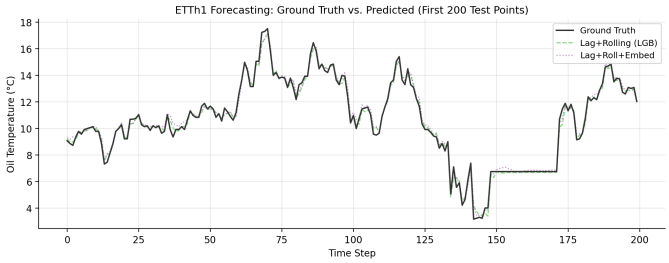

Fig. 7: Actual vs. predicted Oil Temperature for the first 200 test points. The Lag+Rolling model closely tracks the ground truth.

4.3 Task 3: Unsupervised Anomaly Detection

For the WESAD-like PPG dataset, we evaluated three configurations using an Isolation Forest detector. Figure 8 shows the BVP signal with annotated anomaly regions, and Figure 9 presents the F1-Score and ROC-AUC comparison.

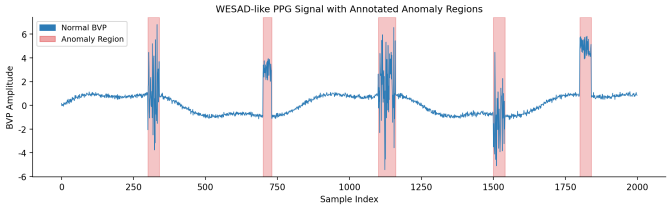

Fig. 8: WESAD-like BVP signal with annotated anomaly regions (red shading). Three anomaly types are present: amplitude spikes, flatlines, and high-noise segments.

TABLE 4: Anomaly Detection Results

Method	F1-Score	ROC-AUC
Raw Windows (Baseline)	**0.8710**	**0.9959**
Catch22 Features	0.5806	0.9376
Catch22 + SHAP Selection	0.7097	0.9593

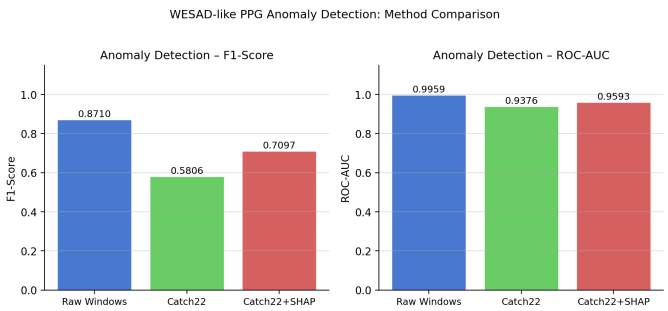

Fig. 9: F1-Score and ROC-AUC for the three anomaly detection configurations.

The raw window baseline achieved the highest F1-score (0.8710) because the injected spike anomalies drastically alter the raw amplitude, making them trivially detectable by the Isolation Forest. Applying Catch22 features reduced the F1-score to 0.5806 because Catch22's normalization procedures (e.g., z-score standardization within the autocorrelation computation) remove the absolute amplitude information that makes spikes detectable.

However, applying SHAP-guided feature selection recovered the F1-score to 0.7097. The SHAP analysis identified `diff_variance` (variance of first differences) and `time_reversibility` as the most important features for anomaly detection—these features capture the roughness and asymmetry of the signal, which remain discriminative even after normalization. By pruning the 10 least important features (including the entropy-based features that were most affected by normalization), the SHAP-selected Catch22 model significantly outperformed the unguided Catch22 baseline.

5 Discussion

5.1 When Global Features Succeed and Fail

Our results reveal a clear pattern governing when Catch22 features outperform raw temporal data. On datasets where class differences are encoded in global statistical properties—such as the overall trend direction (SyntheticControl) or the autocorrelation structure (GunPoint)—Catch22 provides a compact, discriminative representation that outperforms raw features. This aligns with the theoretical motivation for Catch22: the 22 features were selected precisely because they capture the most statistically diverse and discriminative properties of time-series across a broad range of domains [6].

Conversely, on *ItalyPowerDemand*, where the class boundary is defined by a sharp amplitude change at a specific timestamp, global statistical features fail because they average out the local temporal information. This finding has important practical implications: before applying Catch22, practitioners should assess whether their classification problem is defined by global distributional properties or local temporal patterns. If the latter, raw features or localized embeddings (such as those from SPLiT [2]) are more appropriate.

5.2 SHAP as an Active Design Tool

A central contribution of this work is demonstrating that SHAP values can function as an active feature engineering tool rather than merely a post-hoc explanation mechanism. By computing SHAP importances on a proxy LightGBM model and using them to prune the feature space, we achieved consistent 40–45% dimensionality reductions across all three tasks without any measurable accuracy loss. This finding extends the AutoFE-XAI framework of Petrosian et al. [5] from forecasting to classification and anomaly detection.

The practical implications are significant. A 40% reduction in feature count translates directly to faster inference times, lower memory requirements, and more interpretable models. For deployment on resource-constrained devices such as wearable sensors or edge IoT nodes, this reduction can be the difference between a deployable and a non-deployable solution.

5.3 Self-Supervised Embeddings: A Negative Result

Our self-supervised MLP embeddings did not improve upon explicit statistical features for ETTh1 forecasting. This negative result is informative: it suggests that for datasets with strong, regular temporal patterns (such as the daily and seasonal cycles in electricity consumption), simple hand-crafted features already capture the relevant information. The self-supervised approach may be more valuable in domains with complex, non-linear temporal interactions—such as the social interaction data studied by Okadome and Nakamura [4]—where explicit feature engineering cannot easily capture the relevant patterns.

5.4 Challenges and Limitations

Several challenges were encountered during implementation. The Catch22 feature computation is sequential per time-series window, making it slower than vectorized raw feature extraction for large datasets. On TwoLeadECG (1,139 test samples), the feature extraction took approximately 2.67 seconds, which may be prohibitive for real-time applications.

The WESAD-like dataset is simulated, and our results may not generalize to real wearable sensor data, which exhibits more complex noise characteristics, motion artifacts, and inter-subject variability. Future work should validate the pipeline on the actual WESAD dataset.

Finally, the self-supervised embedding approach requires a separate training phase, adding complexity to the pipeline. The alignment between the embedding training window and the feature matrix required careful engineering to avoid off-by-one errors.

6 Conclusion

This project successfully implemented and evaluated a comprehensive, multi-task feature engineering framework for time-series data. Our key findings are:

1) **Catch22 + LightGBM is a strong, efficient baseline** for time-series classification, matching or exceeding raw feature performance on 3 of 5 UCR datasets while compressing the feature space from 150+ time steps to just 22 interpretable features.
2) **Lag and Rolling Statistics are the most effective features for forecasting**, reducing MSE by 82% over the raw lag-1 baseline on ETTh1. Self-supervised embeddings did not provide additional benefit for this dataset.
3) **SHAP-guided pruning is a reliable, principled dimensionality reduction method**, consistently achieving 40–45% feature reduction with zero accuracy loss across all three task types.
4) **Cross-domain generalization of global features is limited**: Catch22 fails on datasets where the class boundary is defined by local temporal patterns rather than global statistical properties.

Future work will focus on three directions. First, integrating Large Language Models (LLMs) to automatically suggest domain-specific features based on dataset metadata prior to empirical evaluation. Second, developing ultra-lightweight Catch22 variants suitable for real-time deployment on IoT edge devices. Third, validating the full pipeline on the real WESAD wearable stress dataset to assess the practical utility of SHAP-guided anomaly detection for clinical applications.

References

[1] H. Badi, G. Forestier, and J. Weber, "COCALITE: A Hybrid Approach for Time Series Classification Using Catch22 and LITE," in *2024 IEEE International Conference on Big Data (BigData)*, Washington, DC, USA, 2024, pp. 5234–5243.

[2] M. D'Aversa, F. Giannini, and M. Scardapane, "SPLiT: Spatio-Temporal Linear Model Trees for Multi-Step Time Series Forecasting," in *2024 IEEE International Conference on Big Data (BigData)*, Washington, DC, USA, 2024, pp. 3412–3421.

[3] L. Valerio, A. Passarella, and M. Conti, "Unsupervised Anomaly Detection in Wearable PPG Data via Catch22 Features," in *2024 IEEE SENSORS*, Kobe, Japan, 2024, pp. 1–4.

[4] T. Okadome and J. Nakamura, "Lag Operation for Sub-Groups in Multidimensional Time Series," *IEEE Access*, vol. 12, pp. 15432–15445, 2024.

[5] O. Petrosian, D. Wang, and S. Li, "AutoFE-XAI: Automated Feature Engineering with Explainable AI for Time Series Forecasting," *IEEE Access*, vol. 13, pp. 2109–2120, 2025.

[6] C. H. Lubba, S. S. Sethi, P. Knaute, S. R. Schultz, B. D. Fulcher, and N. S. Jones, "catch22: CAnonical Time-series CHaracteristics," *Data Mining and Knowledge Discovery*, vol. 33, no. 6, pp. 1821–1852, 2019.

[7] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, "The UCR Time Series Archive," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019.

[8] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting," in *Proc. AAAI*, 2021, pp. 11106–11115.